Part 2: Exploring Classification Algorithms

Data Loading

1. Missing Data Processing and Data Normalization

1.a DecisionTree evaluation of imputation methods

Algorithm	Hyperparam	Value	accuracy	precision	recall	f1
Decision Tree	imputation	baseline	0.9398	0.9419	0.9398	0.9396
		mean	0.9323	0.9358	0.9323	0.9323
		median	0.9323	0.9358	0.9323	0.9323
		full mean	0.9244	0.9276	0.9244	0.9242
		full median	0.9262	0.929	0.9262	0.9259

Evaluation of imputation methods

For evaluation, the evaluation metric was set to "entropy" for information gain, and max_depth is None to give the tree the best chance at success. Decision Trees are very tolerant of missing data. When using cross validation, both median and mean imputation produced a tree that performed worse than baseline.

Adding in median and mean cross validation using the full dataset, we see that median works slightly better than mean, which for a decision tree would make sense. It puts the missing value in the middle of the data, ignoring outliers.

1.b Standardization

Algorithm	Hyperparam	Value	accuracy	precision	recall	f1
Decision Tree	normalization	baseline	0.9323	0.9358	0.9323	0.9323
		standardized	0.9323	0.9358	0.9323	0.9323
		min_max	0.9323	0.9358	0.9323	0.9323
		full standardized	0.9262	0.929	0.9262	0.9259
		full min_max	0.9262	0.929	0.9262	0.9259
KNN	normalization	baseline	0.9373	0.9385	0.9373	0.9362
		standardized	0.9598	0.9614	0.9598	0.9593
		min-max	0.9624	0.9637	0.9624	0.9619
		full standardized	0.9666	0.9675	0.9666	0.9664
		full min-max	0.9701	0.9706	0.9701	0.97

Evaluation of normalization methods

There is no benefit to standardization over baseline in decision trees for either method. This makes sense because the tree is splitting at a threshold, so the number of thresholds doesn't change with scale changes.

KNN shows an improvement using min-max, and this continues through to the full dataset. This implies that the data isn't normally distributed, with multiple modes. Standardization provides additional bits to the data values close to the mean. MinMax scaling provides the same number of bits over the address range. If there are multiple modes at either end of the distribution, MinMax will allow them to be more easily differentiated, where standardization might lose too many digits.

2. Classifier Exploration and Hyperparameter Tuning

Algorithm	Hyperparam	Value	accuracy	precision	recall	f1
KNN	n_neighbours	3	0.9649	0.9668	0.9649	0.9646
		9	0.9708	0.9721	0.9708	0.9705
		15	0.9708	0.9721	0.9708	0.9705
		21	0.9532	0.9566	0.9532	0.9526
Decision Tree	max_depth	2	0.883	0.8841	0.883	0.8815
		8	0.9415	0.9445	0.9415	0.9407
		14	0.9415	0.9445	0.9415	0.9407
		None	0.9415	0.9445	0.9415	0.9407
AdaBoost	n_estimators	10	0.9591	0.9602	0.9591	0.9588
		20	0.9532	0.9549	0.9532	0.9528
		30	0.9649	0.9656	0.9649	0.9647
		100	0.9649	0.9656	0.9649	0.9647
Random Forest	n_estimators	10	0.9298	0.931	0.9298	0.9292
		30	0.9474	0.9483	0.9474	0.947
		50	0.9649	0.9668	0.9649	0.9646
		60	0.9591	0.9616	0.9591	0.9586
		100	0.9532	0.9566	0.9532	0.9526

Observe and discuss the impact of varying the hyperparameter

This felt a little dirty, evaluating the hyperparameter against the test data. We can explain the results afterwards, but predicting them is difficult. All algorithms other than KNN increased in performance as the hyperparameter was increased. This makes sense as the non-KNN algorithms will add more information as the parameter is increased. I added N=100 and max_depth=None to see if there were elbows where performance dropped off. Changing the hyperparameters had very small impacts on overall performance, less than 1% for KNN, 2% for AdaBoost, 3.5% for Random Forest. Decision Tree showed the largest swing at 6%.

AdaBoost performed best at N=30. AdaBoost learns which instances are classified improperly, weighting them in the next iteration. As more iterations are performed, the error term will naturally decrease. The algorithm is able to continue to decrease the error as more cpu is expended. However, each iteration will also add another decision tree to the final model.

RandomForest was second best with n = 50, outperforming the regular decision tree. The random subset used for each tree allow features which are unimportant to have a chance to be ignored. However, as the number of trees increases, performance dropped back down.

Decision Tree hit peak performance at max_depth=8. This is likely because there were no further splits in the training data. Given the 539 instances, a 70% training split = 377, which is smaller than the size of a fully populated tree at 2^8 * 2 = 512 nodes. The minimum leaf size won't allow more nodes. Decision Trees do seem to have a minimum required number of nodes, somwhere between 10 and 20 for this dataset.

KNN peaked at n = 3. Increasing the number of neighbours decreased performance. This is likely because the correct n to use is related to the number of instances in the dataset, and the number of expected clusters.

This shows that algorithms that are able to improve their error term with additional iterations will outperform algorithms that increase the amount of data considered.

3. Feature Engineering and Dimensionality Reduction

3.a Pearson Correlation

Step 1 - use normalised data

Step 2 - translate Y to binary

Step 3 - run the regression

Step 4 - Iterate over the list, and find the indices that are > 0.6

Step 5 - Produce output for report

Step 5.a - Generate the 2d correlation matrix using Pandas - otherwise unused

Correlation Matrix:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	perimeter_se	area_se	smoothness_se	compactness_se	concavity_se	concave points_se	symmetry_se
radius_mean	1.00	0.39	1.00	0.99	0.16	0.53	0.65	0.81	0.17	-0.31	0.70	-0.08	0.70	0.76	-0.20	0.21	0.15	0.32	-0.12
texture_mean	0.39	1.00	0.39	0.38	-0.05	0.26	0.34	0.35	0.08	-0.11	0.33	0.36	0.33	0.32	-0.03	0.19	0.13	0.18	-0.00
perimeter_mean	1.00	0.39	1.00	0.99	0.19	0.58	0.69	0.84	0.20	-0.26	0.71	-0.07	0.72	0.77	-0.18	0.25	0.18	0.35	-0.09
area_mean	0.99	0.38	0.99	1.00	0.17	0.53	0.67	0.82	0.18	-0.27	0.75	-0.06	0.74	0.82	-0.15	0.22	0.17	0.33	-0.09
smoothness_mean	0.16	-0.05	0.19	0.17	1.00	0.64	0.49	0.53	0.58	0.58	0.30	0.09	0.31	0.24	0.32	0.31	0.23	0.36	0.22
compactness_mean	0.53	0.26	0.58	0.53	0.64	1.00	0.88	0.83	0.59	0.55	0.53	0.06	0.60	0.50	0.14	0.73	0.54	0.62	0.24
concavity_mean	0.65	0.34	0.69	0.67	0.49	0.88	1.00	0.91	0.52	0.36	0.66	0.13	0.70	0.64	0.14	0.68	0.69	0.70	0.22
concave points_mean	0.81	0.35	0.84	0.82	0.53	0.83	0.91	1.00	0.47	0.18	0.73	0.07	0.75	0.72	0.06	0.49	0.41	0.59	0.11
symmetry_mean	0.17	0.08	0.20	0.18	0.58	0.59	0.52	0.47	1.00	0.45	0.32	0.12	0.33	0.26	0.17	0.40	0.34	0.36	0.45
fractal_dimension_mean	-0.31	-0.11	-0.26	-0.27	0.58	0.55	0.36	0.18	0.45	1.00	0.00	0.19	0.06	-0.09	0.37	0.55	0.48	0.40	0.35
radius_se	0.70	0.33	0.71	0.75	0.30	0.53	0.66	0.73	0.32	0.00	1.00	0.22	0.97	0.95	0.17	0.39	0.34	0.55	0.20
texture_se	-0.08	0.36	-0.07	-0.06	0.09	0.06	0.13	0.07	0.12	0.19	0.22	1.00	0.22	0.12	0.38	0.26	0.25	0.30	0.40
perimeter_se	0.70	0.33	0.72	0.74	0.31	0.60	0.70	0.75	0.33	0.06	0.97	0.22	1.00	0.93	0.16	0.46	0.37	0.59	0.22
area_se	0.76	0.32	0.77	0.82	0.24	0.50	0.64	0.72	0.26	-0.09	0.95	0.12	0.93	1.00	0.07	0.32	0.27	0.44	0.09
smoothness_se	-0.20	-0.03	-0.18	-0.15	0.32	0.14	0.14	0.06	0.17	0.37	0.17	0.38	0.16	0.07	1.00	0.39	0.32	0.42	0.41
compactness_se	0.21	0.19	0.25	0.22	0.31	0.73	0.68	0.49	0.40	0.55	0.39	0.26	0.46	0.32	0.39	1.00	0.79	0.76	0.41
concavity_se	0.15	0.13	0.18	0.17	0.23	0.54	0.69	0.41	0.34	0.48	0.34	0.25	0.37	0.27	0.32	0.79	1.00	0.80	0.36
concave points_se	0.32	0.18	0.35	0.33	0.36	0.62	0.70	0.59	0.36	0.40	0.55	0.30	0.59	0.44	0.42	0.76	0.80	1.00	0.36
symmetry_se	-0.12	-0.00	-0.09	-0.09	0.22	0.24	0.22	0.11	0.45	0.35	0.20	0.40	0.22	0.09	0.41	0.41	0.36	0.36	1.00
fractal_dimension_se	-0.06	0.03	-0.02	-0.03	0.25	0.47	0.47	0.25	0.31	0.68	0.22	0.30	0.25	0.12	0.45	0.79	0.76	0.67	0.36
radius_worst	0.97	0.41	0.97	0.97	0.20	0.55	0.67	0.82	0.20	-0.25	0.75	-0.10	0.73	0.80	-0.21	0.20	0.14	0.30	-0.13
texture_worst	0.36	0.92	0.37	0.35	0.02	0.28	0.32	0.34	0.10	-0.08	0.25	0.38	0.26	0.27	-0.11	0.14	0.07	0.08	-0.08
perimeter_worst	0.97	0.42	0.97	0.97	0.23	0.61	0.71	0.85	0.23	-0.20	0.75	-0.09	0.76	0.80	-0.19	0.26	0.18	0.34	-0.11
area_worst	0.94	0.40	0.94	0.97	0.20	0.53	0.66	0.81	0.20	-0.22	0.78	-0.07	0.77	0.85	-0.16	0.20	0.15	0.31	-0.11
smoothness_worst	0.15	0.05	0.18	0.15	0.79	0.54	0.42	0.45	0.42	0.45	0.17	-0.07	0.18	0.16	0.29	0.19	0.13	0.19	-0.00
compactness_worst	0.45	0.32	0.49	0.43	0.45	0.88	0.75	0.68	0.48	0.43	0.34	-0.07	0.42	0.35	-0.05	0.66	0.44	0.44	0.08
concavity_worst	0.53	0.33	0.56	0.52	0.41	0.81	0.89	0.76	0.46	0.35	0.44	-0.01	0.49	0.44	-0.02	0.63	0.65	0.57	0.11
concave points_worst	0.74	0.35	0.76	0.73	0.47	0.81	0.86	0.91	0.43	0.19	0.58	-0.05	0.62	0.59	-0.06	0.46	0.40	0.56	0.01
symmetry_worst	0.20	0.12	0.22	0.18	0.39	0.50	0.40	0.37	0.69	0.28	0.13	-0.14	0.14	0.13	-0.12	0.24	0.18	0.09	0.40
fractal_dimension_worst	0.04	0.12	0.09	0.04	0.47	0.69	0.53	0.40	0.44	0.75	0.08	-0.04	0.13	0.06	0.06	0.58	0.44	0.34	0.09
diagnosis	0.74	0.46	0.75	0.73	0.33	0.59	0.67	0.77	0.33	-0.05	0.61	0.01	0.60	0.59	-0.04	0.28	0.22	0.38	-0.00

Number of original features: 30

Number retained features: 12

Algorithm	Hyperparam	Value	accuracy	precision	recall	f1
Decision Tree	feature selection	baseline	0.9415	0.9445	0.9415	0.9407
		pearson	0.9415	0.9419	0.9415	0.9412

Pearson Correlation Performance Evaluation

Using Pearson Correlation for feature selection reduced the number of features from 30 to 12. It had minimal impact on the resulting performance. The drop in precision was a single mis-predicted positive. Looking at the depth of the resulting trees, the baseline tree had a max depth of 5, while the filtered tree had a depth of 8. A shallower tree will perform better - fewer operations to classify an instance.

3.b Feature Extraction using PCA

Algorithm	Hyperparam	Value	accuracy	precision	recall	f1
Baseline Decision Tree	max_depth	2	0.883	0.8841	0.883	0.8815
		8	0.9415	0.9445	0.9415	0.9407
		None	0.9415	0.9445	0.9415	0.9407
PCA Decision Tree	max_depth	2	0.9123	0.9136	0.9123	0.9113
		8	0.9415	0.9415	0.9415	0.9415
		None	0.9415	0.9415	0.9415	0.9415

PCA Performance Evaluation

PCA performs similarly to the baseline tree. Both peak at the same values. However, PCA performs better at the minimum tree size. It might be interesting to see how PCA + AdaBoost perform. The baseline tree has a higher precision, indicating fewer false positives. The initial peak hints that since the features are orthogonal, more information is available earlier in the tree.

4. Comparison: PCA vs Feature Selection

Algorithm	Hyperparam	Value	accuracy	precision	recall	f1
Decision Tree	feature selection	baseline	0.9415	0.9445	0.9415	0.9407
		pearson	0.9415	0.9419	0.9415	0.9412
		pca	0.9357	0.9364	0.9357	0.9352

PCA vs Feature Selection Evaluation

PCA feature selection performed worse than Pearson or the baseline tree. While we saw that the PCA dimensions do encode the same information, this hints that the existing dimensions encode the information more tightly than PCA does. If there are "unused" features in the original dataset, PCA will use them for data. If those dimensions are then dropped, that will reduce the quality of the model.

Once the data is transformed using PCA, it is no longer interpretable. It is very difficult to take a path through the tree and match it back to features in the original dataset, since each PCA feature is multiple original features.

5. K-Fold Cross Validation

Step 1 - use normalised data

Step 2 - create f1 and accuracy accumulators

Step 3 - run knn using cross_validate

Step 4 - run decision tree using cross_validate

Step 5 - produce report

Accuracy

	KNN (k=3)	KNN (k=9)	Decision Tree (max_depth=2)	Decision Tree (max_depth=8)
0	0.950000	0.925000	0.850000	0.825000
1	0.900000	0.925000	0.800000	0.875000
2	0.975000	0.975000	0.900000	0.950000
3	0.950000	0.950000	0.950000	0.900000
4	1.000000	0.975000	0.875000	0.925000
5	0.950000	0.975000	0.950000	0.975000
6	0.975000	0.975000	0.925000	0.925000
7	0.975000	0.950000	0.900000	0.950000
8	0.974359	0.974359	0.897436	1.000000
9	0.948718	1.000000	0.948718	0.923077
Mean	0.959808	0.962436	0.899615	0.924808
Std Dev	0.026891	0.024260	0.048452	0.050004

F1 Score

	KNN (k=3)	KNN (k=9)	Decision Tree (max_depth=2)	Decision Tree (max_depth=8)
0	0.950000	0.922545	0.847009	0.826302
1	0.898006	0.922545	0.796011	0.870909
2	0.974773	0.974773	0.895238	0.949003
3	0.949176	0.949176	0.950000	0.900000
4	1.000000	0.974814	0.874070	0.925444
5	0.950000	0.974814	0.949176	0.974814
6	0.974814	0.975148	0.925444	0.925444
7	0.974814	0.949176	0.900000	0.950000
8	0.974539	0.974539	0.899387	1.000000
9	0.948718	1.000000	0.949359	0.923618
Mean	0.959484	0.961753	0.898569	0.924553
Std Dev	0.027393	0.025182	0.049776	0.050111

Discuss Model Stablility and Reliability

KNN had a much more consistent result, with a standard deviation of 2.5 at K=9. The Decision Tree, with a standard deviation of ~5, had a much wider spread of results. This hints that KNN will generalize better than the Decision Tree.